Тема: Решение задачи анализа тональности (на русском языке)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ..... 5

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ АНАЛИЗА ТОНАЛЬНОСТИ ТЕКСТА.. 8

1.1 Постановка задачи анализа тональности и области применения 8

1.2 Уровни анализа тональности: документ, предложение, аспект..... 9

1.3 Классические подходы к классификации тональности..... 10

1.4 Нейросетевые методы и архитектура Transformer..... 12

1.5 Особенности обработки русскоязычного текста..... 16

1.6 Предобученные языковые модели для русского языка 18

2 МЕТОДОЛОГИЯ РЕШЕНИЯ ЗАДАЧИ 21

2.1 Описание набора данных и распределение классов тональности ... 21

2.2 Предварительная обработка текстовых данных 22

2.3 Токенизация и подготовка входных последовательностей 23

2.4 Разбиение данных и формирование выборок 24

2.5 Стратегия дообучения предобученных моделей..... 25

2.6 Метрики оценки качества классификации 27

2.7 План вычислительных экспериментов 28

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ И АНАЛИЗ РЕЗУЛЬТАТОВ..... 31

3.1 Используемые программные средства и вычислительная среда 31

3.2 Реализация подготовки и токенизации данных 32

3.3 Реализация обучения моделей и пайплайна экспериментов 32

3.4 Сравнение готовых предобученных моделей без дообучения 33

3.5 Проведённые эксперименты по дообучению..... 34

3.6 Анализ процесса обучения, сходимости и ранней остановки..... 36

3.7 Сравнение результатов экспериментов и выбор лучшей модели 39

3.8 Анализ ошибок классификации и матрица ошибок 41
ЗАКЛЮЧЕНИЕ 45
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 46
ПРИЛОЖЕНИЕ 49

ВВЕДЕНИЕ

В современном цифровом пространстве объём текстовой информации, генерируемой пользователями, растёт экспоненциально. Социальные сети, отзывы о товарах и услугах, комментарии на новостных порталах, обращения в службы поддержки, форумы и мессенджеры ежедневно производят миллиарды текстовых сообщений. Значительная часть этих данных содержит субъективные суждения, мнения и эмоциональные оценки, представляющие большую ценность для бизнеса, государственных структур и научных исследований. Ручная обработка таких объёмов информации физически невозможна, что обуславливает потребность в автоматических методах извлечения и классификации тональности текста.

Анализ тональности - одна из ключевых задач обработки естественного языка (Natural Language Processing, NLP), направленная на автоматическое определение эмоционально-оценочной окраски текста. Результаты решения этой задачи находят широкое применение в маркетинговой аналитике (мониторинг отношения потребителей к бренду), банковском секторе (оценка обратной связи клиентов), социологии (анализ общественного мнения), системах рекомендаций и службах автоматической поддержки пользователей.

Несмотря на значительный прогресс в области анализа тональности для английского языка, задача классификации тональности русскоязычного текста остаётся менее изученной. Русский язык характеризуется развитой морфологией, свободным порядком слов, богатой синонимией и большим количеством идиоматических выражений, что усложняет применение классических подходов NLP. Дополнительные сложности создают такие явления, как ирония, сарказм, отрицание и контекстная зависимость оценочной лексики.

Революционным шагом в развитии методов NLP стало появление архитектуры Transformer и предобученных языковых моделей на её основе — BERT, RoBERTa, GPT и их производных. Для русского языка были созданы специализированные варианты: RuBERT, ruRoBERTa, DistilRuBERT и другие. Применение этих моделей в задачах классификации тональности позволяет

существенно повысить качество по сравнению с классическими методами, однако требует тщательного выбора архитектуры, стратегии дообучения и гиперпараметров.

Актуальность работы обусловлена растущей потребностью в эффективных инструментах автоматического анализа тональности русскоязычных текстов, а также необходимостью систематического изучения влияния различных гиперпараметров обучения на качество классификации в условиях ограниченных вычислительных ресурсов.

Объект исследования - методы и алгоритмы автоматической классификации тональности текстовой информации на естественном языке. Предмет исследования - модели и подходы к решению задачи многоклассовой классификации тональности русскоязычного текста на основе предобученных трансформерных архитектур.

Цель работы - разработка и экспериментальное исследование системы классификации тональности русскоязычного текста на основе дообучения предобученных трансформерных моделей, а также определение влияния гиперпараметров обучения на качество классификации.

Для достижения поставленной цели сформулированы следующие задачи:

1. Провести анализ существующих подходов и моделей для решения задачи анализа тональности текста.

2. Изучить особенности применения трансформерных архитектур для классификации русскоязычного текста.

3. Подготовить и предобработать набор данных для трёхклассовой классификации тональности (негативная, нейтральная, позитивная).

4. Реализовать пайплайн дообучения предобученных моделей (RuBERT-tiny2, DistilRuBERT) на выбранном датасете.

5. Спроектировать абляционное исследование для оценки влияния ключевых гиперпараметров: скорости обучения, типа планировщика learning rate, коэффициента регуляризации и размера батча.

6. Провести серию вычислительных экспериментов и собрать метрики качества и времени обучения.

7. Выполнить сравнительный анализ результатов, выявить лучшую конфигурацию и проанализировать ошибки классификации.

Научная и практическая значимость работы заключается в систематизации подходов к дообучению трансформерных моделей для русскоязычного анализа тональности, а также в получении количественных оценок влияния отдельных гиперпараметров на итоговое качество классификации, что позволяет формулировать обоснованные рекомендации по построению подобных систем.

1 Теоретические основы анализа тональности текста

1.1 Постановка задачи анализа тональности и области применения

Задача анализа тональности относится к классу задач классификации текста. Формально её можно сформулировать следующим образом.

Пусть задано множество текстовых документов $D = \{d_1, d_2, \dots, d_N\}$ и конечное множество классов тональности $C = \{c_1, c_2, \dots, c_K\}$. Требуется построить отображение

$$f: D \to C, \tag{1}$$

которое для произвольного текста $d \in D$ возвращает класс $c \in C$, соответствующий его тональной окраске.

В зависимости от мощности множества C выделяют несколько постановок задачи:

- бинарная классификация ($K = 2$): множество $C = \{\text{negative,positive}\}$. Применяется, когда нейтральные тексты отсутствуют или не интересуют исследователя;
- трёхклассовая классификация ($K = 3$): $C = \{\text{negative,neutral,positive}\}$. Наиболее распространённая постановка, в том числе в данной работе;
- пятиклассовая (или порядковая) классификация ($K = 5$): соответствует, например, оценке от 1 до 5 звёзд. Является задачей ординальной классификации, поскольку классы упорядочены;
- регрессионная постановка: вместо дискретного класса предсказывается непрерывная оценка тональности на отрезке $[-1,1]$ или $[0,1]$.

С точки зрения вероятностной интерпретации задача сводится к моделированию условного распределения $P(c \mid d)$ и выбору наиболее вероятного класса:

$$\hat{c} = \arg\max_{c \in C} P\,(\,c \mid d\,). \tag{2}$$

Анализ тональности проводится на нескольких уровнях гранулярности:

Таблица 1 – уровни анализа тональности

Уровень	**Описание**	**Пример задачи**
Документ	Один документ — один класс тональности	Классификация отзыва о фильме
Предложение	Каждое предложение получает свой класс	Анализ новостных лент
Аспектный (ABSA)	Тональность относительно конкретного аспекта объекта	«Камера хорошая, но батарея слабая»
Целевой (targeted)	Тональность относительно конкретной упомянутой сущности	Отношение к политику в твите

В рамках настоящей работы рассматривается классификация тональности на уровне документа.

Основные сферы применения систем анализа тональности:

1. Маркетинг и репутационная аналитика — мониторинг отношения потребителей к бренду, продуктам, рекламным кампаниям.

2. Финансовый сектор — анализ новостей и социальных сетей для прогнозирования настроений на рынке.

3. Клиентская поддержка — автоматическая приоритизация негативных обращений, маршрутизация тикетов.

4. Социология и политология — анализ общественного мнения по социально значимым вопросам.

5. Системы рекомендаций — учёт эмоциональной окраски пользовательских отзывов.

6. Контент-модерация — выявление токсичных и негативных сообщений в социальных платформах.

1.2 Уровни анализа тональности: документ, предложение, аспект

Рассмотрим подробнее различия между уровнями анализа, поскольку выбор уровня определяет архитектуру модели и формат входных данных.

Документ-уровень предполагает, что весь текст выражает единственное настроение. Такой подход уместен для коротких и однородных текстов: отзывов о товарах, твитов, комментариев. Преимущество — простота постановки и реализации; недостаток — потеря информации в случае многоаспектных или смешанных по тональности документов.

Предложение-уровень более гибок: каждый сегмент текста классифицируется независимо. При этом возникает дополнительная сложность — необходимость учитывать контекст соседних предложений, а также обрабатывать предложения без явной тональности.

Аспектный анализ — наиболее сложная и информативная постановка. Текст разбивается на пары (a, s), где a — аспект (свойство объекта), s — тональность относительно него. Например, для фразы «Экран яркий, но звук тихий» аспекты — «экран» (positive) и «звук» (negative). ABSA обычно решается как двухэтапная задача: извлечение аспектов (NER-подобная подзадача) и классификация тональности по аспекту.

В данной работе используется классификация на уровне документа, так как: большинство публично доступных русскоязычных датасетов размечены именно так; задача согласуется с типовыми сценариями применения (фильтрация отзывов) и данная постановка является более простой, что позволяет сфокусироваться на исследовании влияния гиперпараметров.

1.3 Классические подходы к классификации тональности

До появления глубоких нейронных сетей задача анализа тональности решалась классическими методами машинного обучения, которые можно разделить на две группы: лексические подходы и методы машинного обучения с признаками.

Лексические методы основаны на использовании заранее составленных тональных словарей, в которых каждому слову или словосочетанию присвоена оценка тональности. Для русского языка известны словари RuSentiLex, LinisCrowd, RuSentiment. Итоговая тональность документа вычисляется как агрегированная сумма тональностей входящих в него слов:

$$\text{score}(d) = \sum_{w \in d} \text{polarity}(w), \tag{3}$$

где $\text{polarity}(w)$ — оценка тональности слова w из словаря (например, +1 для позитивных, -1 для негативных, 0 для нейтральных).

Преимуществами данных методов является интерпретируемость, отсутствие необходимости в размеченных данных, простота реализации. Недостатками — невозможность учёта контекста, отрицаний, иронии; зависимость от полноты словаря; плохая работа с разговорной лексикой.

Методы машинного обучения с ручными признаками рассматривает анализ тональности как стандартную задачу классификации. Текст представляется в виде вектора признаков, на котором обучается классификатор (наивный байесовский классификатор, метод опорных векторов, логистическая регрессия, градиентный бустинг).

Наиболее распространённые способы векторизации:

1. Bag-of-Words (BoW) — вектор частот слов в документе:

$$x_d = \left(\text{count}(w_1, d), \text{count}(w_2, d), \ldots, \text{count}\left(w_{|V|}, d\right)\right), \tag{4}$$

где V — словарь корпуса.

2. TF-IDF (Term Frequency — Inverse Document Frequency) — взвешенное представление, учитывающее важность слова в корпусе:

$$\text{tfidf}(w, d) = \text{tf}(w, d) \cdot \log \frac{N}{\text{df}(w)}, \tag{5}$$

где $\text{tf}(w, d)$ — частота слова в документе, N — общее число документов, $\text{df}(w)$ — число документов, содержащих слово w.

3. N-граммы — расширение BoW/TF-IDF до последовательностей из n слов или символов, что позволяет частично учитывать порядок слов и устойчивые сочетания.

4. Word2Vec, GloVe, FastText — плотные векторные представления слов, обученные на основе совместной встречаемости. Документ представляется как усреднение векторов входящих в него слов.

Ограничения классических подходов:

- Невозможность учёта длинных контекстных зависимостей.
- Слабая обработка отрицаний («не плохой» становится позитивным).
- Отсутствие моделирования полисемии: одно слово имеет один вектор независимо от контекста.
- Зависимость качества от ручной инженерии признаков.

Эти ограничения преодолеваются нейросетевыми подходами и, в особенности, моделями на базе архитектуры Transformer.

1.4 Нейросетевые методы и архитектура Transformer

Развитие нейросетевых методов обработки естественного языка прошло несколько этапов:

1. Рекуррентные нейронные сети (RNN, LSTM, GRU) — позволили моделировать последовательные зависимости в тексте. Главные ограничения: проблема исчезающих градиентов, последовательная (не параллелизуемая) обработка, ограниченная способность к моделированию длинных зависимостей.

2. Свёрточные нейронные сети для текста (TextCNN) — применяют свёртки по последовательностям эмбеддингов слов для выделения локальных признаков (фактически — нейросетевая аналогия n-грамм).

3. Архитектура Transformer — отказ от рекуррентности в пользу механизма внимания (attention), что обеспечило параллелизуемость и резкий рост качества.

Ключевой компонент Transformer — scaled dot-product attention. Для каждой позиции во входной последовательности вычисляются три вектора:

запрос (Query, Q), ключ (Key, K) и значение (Value, V), полученные линейными проекциями входных эмбеддингов:

$$Q = XW^Q, K = XW^K, V = XW^V, \tag{6}$$

где $X \in R^{n \times d_{\text{model}}}$ — матрица входных представлений, $W^Q, W^K, W^V \in R^{d_{\text{model}} \times d_k}$ — обучаемые матрицы проекций.

Выход attention вычисляется по формуле:

$$\text{Attention}(Q, K, V) = \text{softmax!}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V. \tag{7}$$

Множитель $\frac{1}{\sqrt{d_k}}$ нормализует скалярные произведения, предотвращая попадание softmax в область с очень малыми градиентами при больших d_k.

Для одновременного захвата различных типов зависимостей применяется многоголовое внимание (Multi-Head Attention):

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O, \tag{8}$$

$$\text{head}_i = \text{Attention}\left(QW_i^Q, KW_i^K, VW_i^V\right). \tag{9}$$

Каждая «голова» обучается фокусироваться на своём типе синтаксических или семантических связей.

Полная архитектура одного блока энкодера (Transformer Encoder) состоит из последовательного применения multi-head attention, feed-forward сети, остаточных связей и нормализации слоёв:

$$h' = \text{LayerNorm}\big(x + \text{MultiHead}(x, x, x)\big), \tag{10}$$

$$h = \text{LayerNorm}\big(h' + \text{FFN}(h')\big), \tag{11}$$

где FFN — двухслойная полносвязная сеть с активацией GELU или ReLU:

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2. \tag{12}$$

Поскольку attention инвариантен относительно перестановки входов, в Transformer добавляются позиционные эмбеддинги p_i, кодирующие позицию токена. В оригинальной работе используется синусоидальное кодирование:

$$\text{PE}(pos, 2i) = \sin\Big(\frac{pos}{10000^{2i/d_{\text{model}}}}\Big), \tag{13}$$

$$\text{PE}(pos, 2i+1) = \cos\Big(\frac{pos}{10000^{2i/d_{\text{model}}}}\Big), \tag{14}$$

где pos — позиция токена, i — индекс компонента эмбеддинга, d_{model} — размерность модели. В BERT и его производных используются обучаемые позиционные эмбеддинги — отдельный вектор для каждой позиции, оптимизируемый вместе с остальными параметрами модели.

Итоговое входное представление токена формируется суммой трёх эмбеддингов:

$$x_i = e_{\boldsymbol{token}}(t_i) + e_{\textbf{position}}(i) + e_{\boldsymbol{segment}}(s_i), \tag{15}$$

где $e_{\boldsymbol{token}}$ — эмбеддинг токена, $e_{\textbf{position}}$ — позиционный эмбеддинг, $e_{\boldsymbol{segment}}$ — сегментный эмбеддинг (используется в задачах с двумя предложениями).

В Таблице 2 представлено сравнение архитектур.

Таблица 2 – Преимущества архитектуры Transformer

Свойство	**RNN/LSTM**	**Transformer**
Параллелизация	Последовательная обработка	Полная параллелизация по позициям
Длинные зависимости	Затухание градиентов	Прямой доступ к любой позиции
Вычислительная сложность	$O(n \cdot d^2)$	$O(n^2 \cdot d)$
Интерпретируемость	Скрытые состояния	Веса внимания
Эффективность на GPU	Низкая	Высокая

При длине последовательности n, сравнимой с d_{model}, Transformer оказывается значительно эффективнее RNN на современных ускорителях. Квадратичная сложность по n — известное ограничение, для борьбы с которым разработаны разреженные и линейные модификации внимания (Longformer, Linformer, Performer), не используемые в данной работе.

BERT (Bidirectional Encoder Representations from Transformers) — энкодерная модель на базе Transformer, обучаемая в две стадии:

1. Предобучение (pre-training) на больших неразмеченных корпусах с использованием двух self-supervised задач:

- Masked Language Modeling (MLM): 15 % токенов случайно маскируются, и модель обучается восстанавливать их по контексту. Функция потерь:

$$\mathcal{L}_{MLM} = -\sum_{i \in M} \log P\left(t_i \mid t_{\setminus M}\right), \tag{16}$$

где M — множество замаскированных позиций, $t_{\setminus M}$ — токены вне маски.

- Next Sentence Prediction (NSP): предсказание, следуют ли два предложения друг за другом в исходном тексте. В последующих моделях (RoBERTa) эта задача признана малоэффективной и удалена.

2. Дообучение (fine-tuning) на целевой задаче с размеченными данными: к выходу модели добавляется классификационная «голова», и все параметры

дообучаются совместно с малым learning rate. Для задач классификации последовательностей используется выход специального токена [CLS], проходящий через линейный слой:

$$P(c \mid d) = \text{softmax}(h_{[CLS]}W^{\text{cls}} + b^{\text{cls}}). \quad (17)$$

Парадигма pre-train / fine-tune позволяет переносить языковые знания, выученные на гигантских корпусах, на узкие прикладные задачи, требующие лишь небольшого количества размеченных данных. Именно эта парадигма применяется в настоящей работе.

При дообучении модели на задачу классификации тональности используется категориальная кросс-энтропия:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} y_{i,k} \log \widehat{y_{i,k}}, \quad (18)$$

где N — размер батча, K — число классов, $y_{i,k} \in \{0,1\}$ — индикатор истинного класса (one-hot), $\widehat{y_{i,k}}$ — предсказанная вероятность класса k для примера i, вычисленная через softmax:

$$\widehat{y_{i,k}} = \frac{\exp(z_{i,k})}{\sum_{j=1}^{K}\exp(z_{i,j})}, \quad (19)$$

где $z_{i,k}$ — логит, выходящий из классификационной головы модели для k-го класса.

1.5 Особенности обработки русскоязычного текста

Русский язык обладает рядом лингвистических особенностей, существенно влияющих на построение NLP-систем.

Русский — флективный язык с развитой системой словоизменения. Существительное может иметь до 12 падежно-числовых форм, прилагательное — десятки форм по родам, числам, падежам и степеням сравнения, глагол — несколько десятков форм по лицам, числам, временам, наклонениям, видам. Это приводит к взрывному росту словаря при наивной токенизации по словам. Например, для одной леммы «хороший» в корпусе встречаются формы «хорошая», «хорошее», «хорошие», «хорошего», «хорошему», «хорошим» и т. д.

Классическим решением была лемматизация — приведение слов к начальной форме. В современных трансформерных моделях эта проблема решается subword-токенизацией (BPE, WordPiece, SentencePiece), которая разбивает слово на морфологически значимые подсловные единицы.

В отличие от английского, в русском языке порядок слов в предложении гибкий: «Студент написал курсовую», «Курсовую написал студент», «Написал студент курсовую» — все варианты грамматичны и различаются лишь коммуникативным акцентом. Это снижает эффективность n-граммных подходов и повышает значимость моделей, способных захватывать дальние зависимости, — таких как Transformer.

Отрицание в русском языке может быть выражено множественно (двойное и тройное отрицание считается нормой: «никто никогда ничего не говорил») и инвертирует тональность фразы. Лексические подходы плохо справляются с этим явлением, тогда как контекстные модели типа BERT способны корректно интерпретировать отрицания.

Тексты социальных сетей и отзывов содержат большое количество разговорных форм, орфографических ошибок, латинских вкраплений, эмодзи и неологизмов. Это требует устойчивости моделей к шуму, что обеспечивается subword-токенизацией и предобучением на разнородных корпусах, включающих, в частности, конверсационные данные.

По сравнению с английским, русский язык располагает существенно меньшим объёмом размеченных датасетов для задач NLP. Это делает особенно

ценным использование предобученных моделей, минимизирующих требуемое количество размеченных данных для целевой задачи.

1.6 Предобученные языковые модели для русского языка

Для русскоязычных NLP-задач разработан ряд предобученных моделей, различающихся по архитектуре, размеру, корпусу предобучения и целевой области применения. Рассмотрим основные из них в Таблице 3.

Таблица 3 – Сравнительная характеристика моделей

Модель	**Параметры**	**Слои**	**Скрытая размерность**	**Корпус предобучения**	**Авторы**
RuBERT (base)	~178 M	12	768	Русская Википедия + новости	DeepPavlov
RuBERT-tiny2	~29 M	3	312	Разговорный + общий	cointegrated
DistilRuBERT-small-cased-conversational	~107 M	6	768	Разговорные русскоязычные данные	DeepPavlov
ruRoBERTa-large	~355 M	24	1024	Веб-корпус, книги, новости	SberDevices
ruBERT-large	~427 M	24	1024	Общерусский корпус	SberDevices

RuBERT — адаптация оригинального BERT для русского языка, выполненная коллективом DeepPavlov. Модель инициализирована из мультиязычного BERT (mBERT) и дополнительно обучена на русском корпусе. Использует WordPiece-токенизацию со словарём ~120 K subword-токенов, оптимизированных для русского языка. Базовая модель содержит 12 слоёв с скрытой размерностью 768, что соответствует стандартной BERT-base конфигурации.

RuBERT-tiny2 — компактная модель от автора cointegrated, ориентированная на сценарии с ограниченными вычислительными ресурсами. Содержит всего 3 трансформерных слоя с размерностью 312, что даёт ~29 M параметров — приблизительно в 6 раз меньше, чем BERT-base. Несмотря на

малый размер, модель демонстрирует достойное качество на задачах классификации благодаря дистилляции из более крупной модели и оптимизированной токенизации. Преимущество: высокая скорость инференса, низкое потребление памяти. Недостаток: ограниченная семантическая ёмкость по сравнению с полноразмерными моделями.

DistilRuBERT-small-cased-conversational — дистиллированная версия RuBERT, разработанная DeepPavlov специально для разговорных данных (отзывы, социальные сети). Получена методом knowledge distillation — обучения «ученика» на выходах «учителя». При сохранении ~6 слоёв вместо 12 модель достигает 95–97 % качества полного RuBERT при ~2× ускорении инференса. Особенность: предобучение на конверсационном корпусе делает модель особенно подходящей для задач анализа тональности отзывов и комментариев.

ruRoBERTa-large — крупная модель от SberDevices, основанная на архитектуре RoBERTa (модификация BERT без задачи NSP, с динамическим маскированием и обучением на больших батчах). Содержит 24 слоя и 355 M параметров. Демонстрирует state-of-the-art результаты на многих русскоязычных бенчмарках, но требует значительных вычислительных ресурсов для дообучения.

В данной работе для дообучения выбраны две модели:

1. cointegrated/rubert-tiny2 — представитель класса компактных моделей, позволяющий оценить достижимое качество в условиях ресурсных ограничений.

2. DeepPavlov/distilrubert-small-cased-conversational — модель среднего размера, специализированная на разговорных данных, что особенно релевантно для анализа тональности отзывов и пользовательских комментариев.

В первой главе рассмотрены теоретические основы задачи анализа тональности текста. Проанализирована формальная постановка задачи как задачи многоклассовой классификации, описаны различные уровни анализа и области практического применения. Рассмотрены классические подходы (лексические и основанные на ручных признаках) и их фундаментальные ограничения в условиях морфологически богатого русского языка. Подробно

изложены принципы архитектуры Transformer. Описаны лингвистические особенности русского языка, влияющие на построение NLP-систем, и проведён сравнительный обзор предобученных моделей для русского языка.

2 Методология решения задачи

В данной главе описана методология экспериментального исследования задачи трёхклассовой классификации тональности русскоязычного текста. Изложены характеристики используемого набора данных, процедура его разбиения и предобработки, архитектура классификационной модели на основе предобученных трансформеров, стратегия дообучения, метрики оценки качества и план абляционного исследования влияния гиперпараметров.

2.1 Описание набора данных и распределение классов тональности

В качестве источника данных для экспериментов используется датасет RuReviews — открытый русскоязычный корпус отзывов о товарах, представленный С. Метаниным. Корпус содержит пользовательские отзывы из категории «Женская одежда и аксессуары» крупного российского интернет-магазина и предназначен для решения задачи трёхклассовой классификации тональности. В Таблице 4 представлены характеристики датасета.

Таблица 4 – Характеристики датасета

Параметр	**Значение**
Название	RuReviews (women-clothing-accessories.3-class.balanced)
Источник	Открытый репозиторий sismetanin/rureviews
Домен	Отзывы о товарах (женская одежда и аксессуары)
Язык	Русский
Общий объём	90 000 примеров
Число классов	3 (negative, neutral, positive)
Сбалансированность	Сбалансирован по классам
Формат	CSV, два поля: text, label

Датасет полностью сбалансирован — каждому классу соответствует одинаковое количество примеров. В таблице 5 представлено разделение данных датасета по классам.

Таблица 5 – Распределение данных по классам

Класс	**Идентификатор**	**Количество**	**Доля**
neutral	0	30 000	33.3 %
positive	1	30 000	33.3 %
negative	2	30 000	33.3 %

Сбалансированность датасета — важное преимущество с точки зрения обучения классификатора, поскольку устраняет проблему перекоса предсказаний в сторону мажоритарного класса и позволяет использовать стандартные функции потерь без перевзвешивания. При этом сбалансированный датасет делает метрику macro F1 содержательной: она усредняет F1 по классам с равным весом, что соответствует реальной структуре данных.

2.2 Предварительная обработка текстовых данных

В современных пайплайнах на основе трансформерных моделей с subword-токенизацией традиционные шаги «глубокой» предобработки не применяются, поскольку:

- subword-токенизатор (WordPiece, BPE, SentencePiece) самостоятельно разбивает слова на морфологически значимые единицы, эффективно обрабатывая словоизменение;
- регистр и пунктуация несут информацию о тональности (восклицательные знаки, заглавные буквы как маркер эмоции) и должны сохраняться;
- стоп-слова в задачах анализа тональности могут содержать значимую информацию (отрицания, усилители: «не», «очень», «совсем»).

В рамках работы реализована минимальная предобработка, состоящая из следующих этапов:

1. Загрузка данных из CSV-файла с разделителем табуляции.

2. Обработка пропущенных значений — замена `None`-значений в текстовом поле на пустую строку и приведение всех значений к строковому типу.

3. Передача в токенизатор без дополнительных преобразований. Все этапы нормализации (приведение Unicode, обработка специальных символов, обрезка по длине) выполняются токенизатором соответствующей предобученной модели.

Такой минималистичный подход является современным стандартом при работе с трансформерными моделями и позволяет максимально использовать языковые знания, заложенные на этапе предобучения.

2.3 Токенизация и подготовка входных последовательностей

В предобученных моделях RuBERT-tiny2 и DistilRuBERT используется субсловная токенизация, основанная на алгоритме WordPiece. Метод итеративно строит словарь, выбирая на каждом шаге пару символов или подсловных единиц, объединение которых максимизирует правдоподобие корпуса:

$$\text{score}(a, b) = \frac{\text{count}(ab)}{\text{count}(a) \cdot \text{count}(b)}. \quad (20)$$

Результирующий словарь содержит как полные часто встречающиеся слова, так и морфемы (приставки, корни, суффиксы), что особенно эффективно для морфологически богатого русского языка. Слово, отсутствующее в словаре, разбивается на известные subword-единицы, что устраняет проблему out-of-vocabulary (OOV) слов.

Применённые параметры токенизации для всех моделей представлены в Таблице 6.

Таблица 6 – Параметры токенизации

Параметр	**Значение**	**Обоснование**
truncation	True	Обрезка превышающих лимит последовательностей
max_length	256 токенов	Соответствует типичной длине отзыва; покрывает подавляющее большинство примеров корпуса без обрезки и обеспечивает приемлемую вычислительную сложность (внимание имеет квадратичную сложность по длине)
padding	Динамический (через DataCollator)	Дополнение до длины самой длинной последовательности в батче — экономит вычисления

Каждый токенизированный пример преобразуется в три тензора одинаковой длины:

- input_ids — последовательность индексов токенов в словаре модели. Начинается со специального токена [CLS] и заканчивается [SEP]:

$$x = ([CLS], t_1, t_2, \dots, t_n, [SEP]). \quad (21)$$

- attention_mask — бинарный вектор той же длины: 1 для значимых токенов, 0 для токенов-заполнителей (padding).
- token_type_ids — вектор сегментов; в задаче классификации одиночной последовательности заполнен нулями.

Для оптимизации обучения применяется динамический padding через DataCollatorWithPadding: последовательности в каждом батче дополняются до длины самой длинной последовательности именно в этом батче, а не до глобального `max_length=256`. Это существенно сокращает вычислительные затраты при обработке коротких отзывов и комбинируется с группировкой примеров по длине для дополнительного ускорения.

2.4 Разбиение данных и формирование выборок

Исходный датасет из 90 000 примеров разбивается на обучающую, валидационную и тестовую выборки в соотношении 70:15:15 с применением стратификации по классам, что гарантирует одинаковое распределение классов в обеих выборках. Стратификация реализована вручную через группировку датафрейма по полю label и независимое разбиение каждой группы. В Таблице 7 представлено соотношение выборок.

Таблица 7 – Соотношение выборок

Выборка	**Доля**	**Назначение**
train	70%	Обучение параметров модели
validation	15%	Подбор гиперпараметров, early stopping, выбор лучшего чекпойнта
test	15%	Финальная независимая оценка лучшей модели

Данное соотношение является общепринятым компромиссом между объёмом обучающих данных и надёжностью оценки качества. Большая доля обучающей выборки обеспечивает достаточный объём данных для дообучения трансформерной модели, тогда как 15 % валидации и 15 % теста являются достаточным объёмом для статистически устойчивой оценки метрик качества.

С учётом плана исследования, включающего 10 полных экспериментов (2 модели × 5 конфигураций гиперпараметров), а также ограничений вычислительной среды, для проведения экспериментов применяется ограничение размера выборок.

Выбор подмножества выполняется после случайного перемешивания, что сохраняет стратификацию по классам в силу исходной сбалансированности данных. Урезанный объём выборки является достаточным для дообучения предобученных трансформерных моделей.

2.5 Стратегия дообучения предобученных моделей

Дообучение (fine-tuning) — это процесс адаптации предобученной модели к конкретной прикладной задаче на относительно небольшом размеченном наборе данных. В отличие от обучения «с нуля», при дообучении большинство параметров модели уже инициализированы осмысленными значениями, полученными в ходе предобучения на больших корпусах текста. Это позволяет достичь высокого качества за малое число итераций и при ограниченном объёме данных.

В настоящей работе применяется классическая стратегия полного дообучения, при которой обновляются все параметры модели — как параметры предобученного трансформер-энкодера, так и параметры случайно инициализированной классификационной головы.

В качестве оптимизатора используется AdamW — модифицированная версия популярного оптимизатора Adam, в которой регуляризация весов применяется отдельно от градиентного шага. Такое разделение даёт более корректное поведение регуляризации и стало де-факто стандартом для дообучения трансформерных моделей.

Ключевые гиперпараметры оптимизатора:

- learning rate — задаёт величину шага градиентного спуска. Для дообучения BERT-подобных моделей рекомендуемое значение — 2e-5. Большие значения могут разрушить предобученные представления, малые — приводят к медленной сходимости.

- weight decay — коэффициент регуляризации, ограничивающий рост абсолютных значений весов. Стандартное значение 0.01; в одной из экспериментальных конфигураций исследуется увеличение до 0.1.
- $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 1e\text{-}8$ — стандартные параметры экспоненциального сглаживания градиентов, оставлены по умолчанию.

Для трёхклассовой классификации используется стандартная функция потерь — категориальная кросс-энтропия (Cross-Entropy Loss). Идея функции потерь проста: для каждого примера модель выдаёт вероятности всех трёх классов; функция потерь штрафует модель тем сильнее, чем меньшую вероятность модель присвоила истинному классу. Минимизация средней кросс-энтропии по обучающей выборке эквивалентна обучению модели присваивать высокие вероятности правильным меткам.

Размер батча определяет, сколько примеров обрабатывается за один шаг градиентного спуска. Базовое значение — 16; в одной из конфигураций исследуется увеличение до 32.

- меньший батч даёт более частые обновления параметров, но более «шумную» оценку градиента;
- больший батч даёт более устойчивую оценку градиента, но меньшее число шагов обучения за эпоху.

Число эпох — сколько раз модель проходит по всей обучающей выборке. Максимальное число эпох ограничено восемью, однако реальное число эпох определяется автоматически механизмом ранней остановки.

Чтобы избежать переобучения и не тратить время на бесполезное продолжение обучения, применяется механизм ранней остановки. После каждой эпохи на валидационной выборке вычисляется значение macro F1. Если в течение двух последовательных эпох улучшения не происходит, обучение прерывается, а в качестве итогового состояния модели восстанавливается чекпойнт с лучшим значением macro F1.

2.6 Метрики оценки качества классификации

Для оценки качества трёхклассовой классификации тональности используются следующие метрики. Введём обозначения для класса k:

- TP_k — число истинно-положительных предсказаний (истинный класс k, предсказан k);
- FP_k — число ложноположительных (истинный класс $\neq k$, предсказан k);
- FN_k — число ложноотрицательных (истинный класс k, предсказан $\neq k$);
- TN_k — число истинно-отрицательных.

Accuracy (доля правильных ответов) — доля корректно классифицированных примеров от их общего числа:

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[\widehat{y_i} = y_i] = \frac{\sum_{k=1}^{K} TP_k}{N}. \quad (22)$$

Метрика интуитивно понятна и применима к сбалансированному датасету RuReviews, однако её недостаточно для всестороннего анализа качества — она не различает типы ошибок и не показывает качество по классам.

Precision (точность) — доля корректных предсказаний среди всех предсказаний класса k:

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k}. \quad (23)$$

Recall (полнота) — доля корректно найденных примеров среди всех примеров класса k:

$$\text{Recall}_k = \frac{TP_k}{TP_k + FN_k}. \quad (24)$$

F1-мера — гармоническое среднее precision и recall:

$$F1_k = \frac{2 \cdot \text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k}. \quad (25)$$

Гармоническое среднее, в отличие от арифметического, штрафует за дисбаланс между precision и recall: высокий F1 достигается только при одновременно высоких значениях обеих метрик.

В качестве главной метрики для выбора лучшего чекпойнта в процессе обучения и для итогового сравнения конфигураций используется macro F1. Выбор обусловлен следующими соображениями:

1. Низкое качество хотя бы по одному классу резко снижает macro F1, что соответствует требованию равной важности всех типов тональности.

2. Для задачи классификации отзывов одинаково важно как не пропускать негативные обращения (recall), так и не помечать нейтральные тексты как негативные (precision).

3. Macro F1 является основной метрикой в большинстве работ по анализу тональности, что обеспечивает сопоставимость результатов.

2.7 План вычислительных экспериментов

Экспериментальная часть работы состоит из двух блоков.

В первом блоке производится оценка качества предобученных моделей, опубликованных авторами вместе с классификационной головой для задачи анализа тональности. Список моделей:

- blanchefort/rubert-base-cased-sentiment — RuBERT-base, дообученная для трёхклассовой классификации тональности;
- cointegrated/rubert-tiny-sentiment-balanced — компактная модель, дообученная на сбалансированных данных.

Этот блок служит базовой точкой отсчёта (baseline-эталоном) и позволяет оценить, какой прирост качества даёт самостоятельное дообучение по сравнению с использованием готовых моделей «из коробки».

Основной блок — серия экспериментов по дообучению двух базовых моделей в пяти различных конфигурациях гиперпараметров. Список моделей:

- cointegrated/rubert-tiny2;
- DeepPavlov/distilrubert-small-cased-conversational.

В рамках исследования применяются 5 конфигураций гиперпараметров, каждая из которых запускается на двух базовых моделях (RuBERT-tiny2 и DistilRuBERT-small-cased-conversational), что в сумме даёт 10 экспериментов. Дизайн исследования построен по принципу «один параметр за раз» (one-factor-at-a-time, OFAT): в каждой конфигурации, отличной от baseline, изменяется ровно один гиперпараметр, остальные остаются на baseline-значениях. В Таблице 8 представлены используемые конфигурации гиперпараметров модели.

Таблица 8 – Конфигурации гиперпараметров

Гиперпараметр	**baseline**	**high_lr**	**cosine_sched**	**strong_wd**	**large_batch**
Learning rate	2,00E-05	5,00E-05	2,00E-05	2,00E-05	2,00E-05
Тип scheduler	linear	linear	cosine	linear	linear
Warmup ratio	0.1	0.1	0.1	0.1	0.1
Weight decay	0.01	0.01	0.01	0.1	0.01
Batch size (train)	16	16	16	16	32
Batch size (eval)	32	32	32	32	32
Max sequence length	256	256	256	256	256
Максимум эпох	8	8	8	8	8
Early stopping patience	2	2	2	2	2

Во второй главе была представлена методология решения задачи трёхклассовой классификации тональности русскоязычного текста. Описан используемый набор данных RuReviews — открытый сбалансированный корпус из 90 000 отзывов о товарах. Обоснована процедура минимальной предобработки, согласующаяся с современной практикой использования трансформерных моделей. Описано стратифицированное разбиение данных и параметры токенизации. Представлена общая архитектура классификационной модели на основе предобученного трансформера и линейной классификационной головы, а также детально изложена стратегия дообучения с использованием оптимизатора AdamW, планировщиков learning rate и механизма ранней остановки. Определён набор метрик качества с обоснованием

выбора macro F1 в качестве главной метрики. Сформулирован план абляционного исследования из 10 экспериментов (2 модели × 5 конфигураций), позволяющий изолированно оценить влияние четырёх ключевых гиперпараметров обучения: learning rate, типа планировщика, коэффициента регуляризации и размера батча.

3 Программная реализация и анализ результатов

В данной главе представлены результаты практической части исследования: описаны программные средства и вычислительная среда, реализация подготовки данных и процедуры дообучения, проведено сравнение готовых предобученных моделей, представлены результаты 10 экспериментов по дообучению, выполнен анализ влияния гиперпараметров на качество классификации, проанализирована динамика обучения и ошибки лучшей модели.

3.1 Используемые программные средства и вычислительная среда

Программная реализация выполнена на языке Python 3 с использованием экосистемы библиотек для глубокого обучения и обработки данных. Основные используемые библиотеки представлены в Таблице 9.

Таблица 9 – Используемые библиотеки

Библиотека	**Назначение**
transformers	Загрузка предобученных моделей и токенизаторов, реализация Trainer API
datasets	Хранение, разбиение и предобработка датасета
torch (PyTorch)	Бэкенд глубокого обучения, GPU-вычисления
scikit-learn	Вычисление метрик качества, confusion matrix, classification report
pandas	Работа с табличными данными, формирование сводных таблиц результатов
numpy	Численные операции
matplotlib, seaborn	Визуализация результатов: кривые обучения, матрицы ошибок, столбчатые диаграммы

Использование высокоуровневого Trainer API из библиотеки transformers позволило унифицировать процедуру обучения для всех 10 экспериментов и сосредоточиться на варьировании гиперпараметров, а не на низкоуровневой реализации цикла обучения.

Все эксперименты проведены в облачной среде Google Colab с использованием графического ускорителя NVIDIA Tesla T4 (16 ГБ VRAM, архитектура Turing). Обучение выполнялось в режиме одинарной точности (fp32). Использование единой вычислительной среды для всех 10 экспериментов обеспечивает корректность сравнения времени работы между конфигурациями.

3.2 Реализация подготовки и токенизации данных

Подготовка данных реализована в виде последовательного пайплайна:

1. Загрузка корпуса. Датасет RuReviews загружается напрямую из публичного репозитория средствами pandas.

2. Стратифицированное разбиение на три выборки. Данные разбиваются на train, validation и test в соотношении 70/15/15 с сохранением баланса классов через независимую обработку каждой группы.

3. Преобразование в формат DatasetDict. Полученные pandas.DataFrame оборачиваются в объекты библиотеки datasets:

4. Ограничение объёма выборок. Для каждого эксперимента используются первые 6 000 примеров обучающей и по 1 500 примеров валидационной и тестовой выборок (после перемешивания с фиксированным SEED).

5. Токенизация. Для каждой пары «базовая модель — конфигурация» загружается соответствующий токенизатор и применяется ко всем трём сплитам через метод DatasetDict.map():

6. Динамический padding. Дополнение последовательностей до длины самой длинной в батче выполняется через DataCollatorWithPadding, что в сочетании с параметром group_by_length=True существенно ускоряет обучение.

3.3 Реализация обучения моделей и пайплайна экспериментов

Процедура дообучения инкапсулирована в функцию train_one_experiment(base_model, config), выполняющую следующие шаги:

1. Установка единого случайного зерна через set_seed(SEED).
2. Токенизация выборок для текущего токенизатора.

3. Создание модели с классификационной головой через AutoModelForSequenceClassification.from_pretrained(...) с параметром num_labels=3.
4. Конфигурирование TrainingArguments на основе словаря текущей конфигурации (learning_rate, scheduler_type, weight_decay, per_device_train_batch_size).
5. Инициализация Trainer с пользовательской функцией метрик compute_metrics и callback'ом EarlyStoppingCallback(early_stopping_patience=2).
6. Запуск trainer.train() и сохранение истории trainer.state.log_history.
7. Сохранение итоговых метрик в общий датафрейм результатов.

Общий цикл по парам формирует все 10 экспериментов. Унификация пайплайна гарантирует, что различия в результатах между экспериментами обусловлены исключительно различиями в исследуемых гиперпараметрах, а не в особенностях реализации.

3.4 Сравнение готовых предобученных моделей без дообучения

Перед проведением экспериментов по дообучению выполнена базовая оценка двух готовых моделей, опубликованных авторами вместе с классификационной головой для задачи анализа тональности. Модели применены к валидационной выборке в режиме zero-shot, без какой-либо дополнительной настройки. Результаты экспериментов представлены в Таблице 10.

Таблица 10 – Метрики готовых предобученных моделей

Модель	**Accuracy**	**Macro Precision**	**Macro Recall**	**Macro F1**
blanchefort/rubert-base-cased-sentiment	0.782	0.807	0.788	0.781
cointegrated/rubert-tiny-sentiment-balanced	0.744	0.750	0.747	0.748

Модель blanchefort/rubert-base-cased-sentiment демонстрирует существенно более высокое качество — macro F1 = 0.781. Это объясняется тем, что данная модель основана на полноразмерной RuBERT-base (~178 M

параметров), предобученной на большом русскоязычном корпусе, и затем дообученной авторами на размеченных данных для задачи анализа тональности.

Модель cointegrated/rubert-tiny-sentiment-balanced показывает macro F1 = 0.748 — это компактная модель, предоставляющая компромисс между качеством и вычислительной эффективностью.

Полученные значения являются естественной точкой отсчёта для оценки наших дообученных моделей.

3.5 Проведённые эксперименты по дообучению

В соответствии с планом, описанным в главе 2.7, выполнены 10 экспериментов: 2 базовые модели × 5 конфигураций гиперпараметров. Итоговые метрики на валидационной выборке представлены в Таблице 11, отсортированной по убыванию macro F1.

Таблица 11 – Сводные результаты 10 экспериментов по дообучению

№	Модель	Конфигурация	Accuracy	Macro F1	Macro Prec	Macro Rec	Eval Loss	Эпох
1	distilrubert	**baseline**	0.760	**0.7609**	0.7631	0.7594	0.6005	6
2	distilrubert	strong_regularization	0.759	0.7607	0.7637	0.7588	0.5966	6
3	distilrubert	cosine_scheduler	0.756	0.7572	0.7608	0.7552	0.6090	6
4	distilrubert	high_lr	0.750	0.7513	0.7554	0.7492	0.6103	5
5	rubert-tiny2	large_batch	0.749	0.7495	0.7496	0.7497	0.5916	8
6	rubert-tiny2	strong_regularization	0.749	0.7489	0.7490	0.7491	0.5856	6
7	rubert-tiny2	cosine_scheduler	0.745	0.7460	0.7485	0.7443	0.6041	8
8	distilrubert	large_batch	0.745	0.7451	0.7487	0.7467	0.5835	5
9	rubert-tiny2	baseline	0.740	0.7407	0.7418	0.7399	0.5983	8
10	rubert-tiny2	high_lr	0.738	0.7389	0.7400	0.7379	0.6207	3

Визуальное сравнение всех 10 конфигураций по четырём метрикам приведено на Рисунке 1.

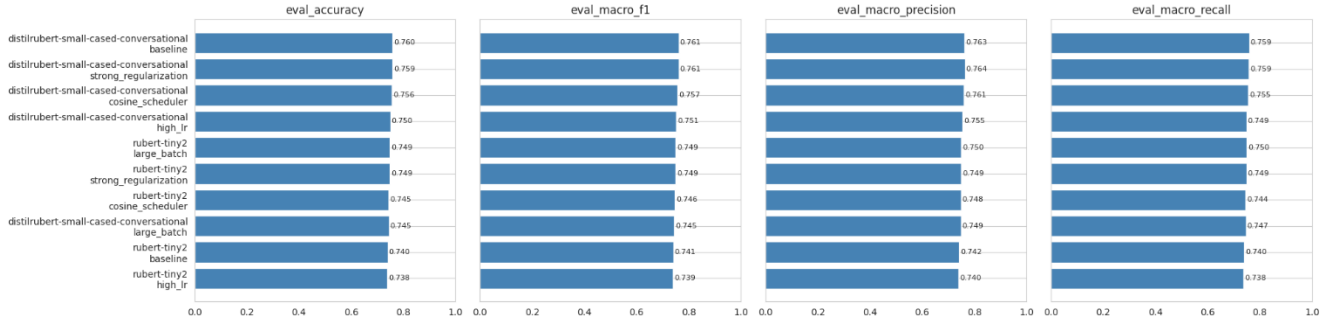


Рисунок 1 — Сравнение метрик качества всех 10 экспериментов

На рисунке представлены столбчатые диаграммы для четырёх метрик качества. Конфигурации отсортированы по убыванию значения соответствующей метрики. Все четыре метрики дают согласованную упорядоченность конфигураций — расхождения минимальны. Верхние четыре позиции во всех метриках занимают конфигурации на базе distilrubert. Модель rubert-tiny2 во всех конфигурациях стабильно уступает distilrubert, но разрыв невелик — около 1–2 процентных пунктов. Диапазон значений всех конфигураций укладывается в узкий интервал 0.738–0.761 по macro F1, что свидетельствует об относительно слабой чувствительности задачи к выбору гиперпараметров в исследуемом диапазоне.

Усредненные результаты по 5 конфигурациям для каждой базовой модели представлены в Таблице 12.

Таблица 12 – Сравнение базовых моделей

Базовая модель	**Средний Macro F1**	**Параметры**
DistilRuBERT-small-cased-conversational	0.7550	~107 M
RuBERT-tiny2	0.7448	~29 M

DistilRuBERT в среднем превосходит RuBERT-tiny2 на ~1 процентный пункт по macro F1. Это согласуется с теоретическими ожиданиями: модель большего размера обладает большей репрезентативной ёмкостью. Дополнительный фактор — специализация DistilRuBERT на разговорных данных, близких по стилю к пользовательским отзывам.

3.6 Анализ процесса обучения, сходимости и ранней остановки

Для понимания поведения моделей в процессе обучения проанализирована динамика метрик по эпохам, представленная на Рисунках 2–4.

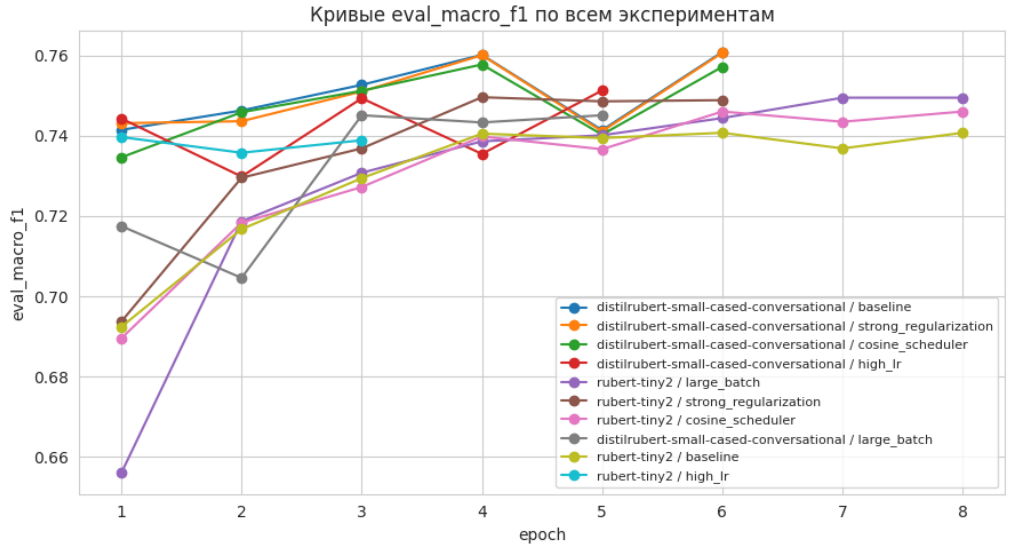


Рисунок 2 — Динамика eval_macro_f1 по эпохам для всех 10 экспериментов

На графике отображены кривые изменения macro F1 для каждой из 10 конфигураций. Можно выделить следующие группы поведения:

- DistilRuBERT-конфигурации (синяя, оранжевая, зелёная, красная кривые) уже на первой эпохе стартуют с относительно высоких значений (0.69–0.74) и быстро выходят на плато 0.75–0.76 к 3–4 эпохе. Дальнейшее обучение даёт лишь незначительные колебания.
- RuBERT-tiny2 / large_batch (фиолетовая кривая) стартует с самого низкого значения (0.656 на первой эпохе) — это связано с уменьшением числа шагов оптимизации при батче 32. Однако модель уверенно растёт на протяжении всех 8 эпох, достигая 0.7495.
- RuBERT-tiny2 / cosine_scheduler и baseline показывают плавный рост на протяжении всех 8 эпох без достижения чёткого плато.
- RuBERT-tiny2 / high_lr (голубая кривая) демонстрирует характерную картину преждевременной остановки: модель достигает 0.74 на первой эпохе, но в течение следующих эпох не улучшается и останавливается на 3-й эпохе по early stopping.

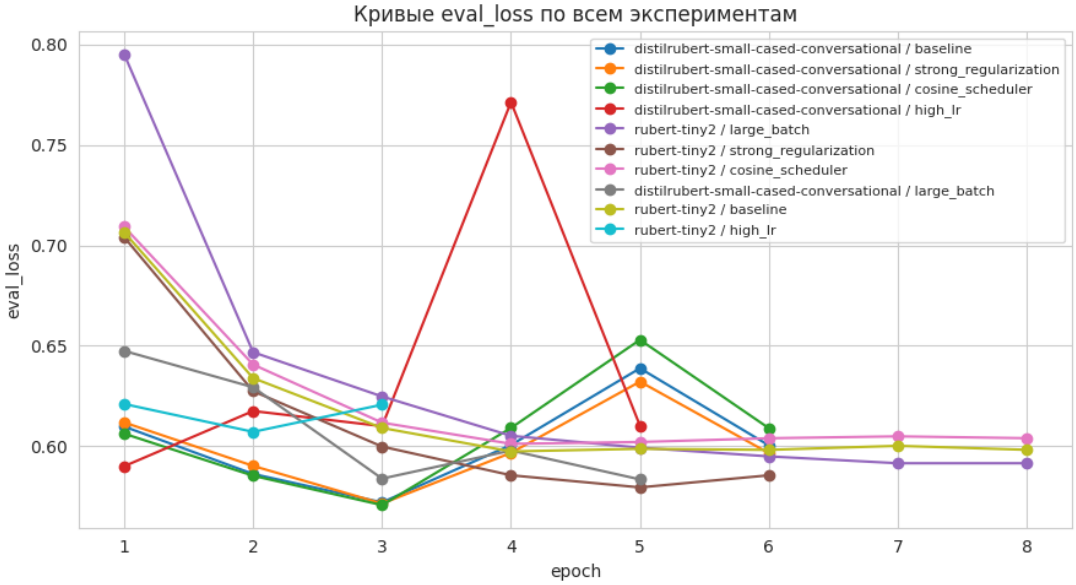


Рисунок 3 — Динамика eval_loss по эпохам для всех 10 экспериментов

На графике видна общая тенденция снижения функции потерь с ~0.70 до ~0.59–0.62, однако с заметными особенностями:

- большинство кривых достигают минимума на 3-й эпохе (~0.58–0.60) и далее показывают слабый рост — классический признак начала переобучения, при котором eval_loss растёт раньше, чем падает целевая метрика;
- distilRuBERT / high_lr (красная кривая) демонстрирует аномальный выброс на 4-й эпохе — eval_loss подскакивает до 0.77, после чего восстанавливается. Это типичный признак нестабильности оптимизации при завышенном learning rate;
- ruBERT-tiny2 / large_batch (фиолетовая кривая) на первой эпохе имеет наибольший loss (0.795), что подтверждает наблюдение об ограниченном числе шагов градиента при большом батче.

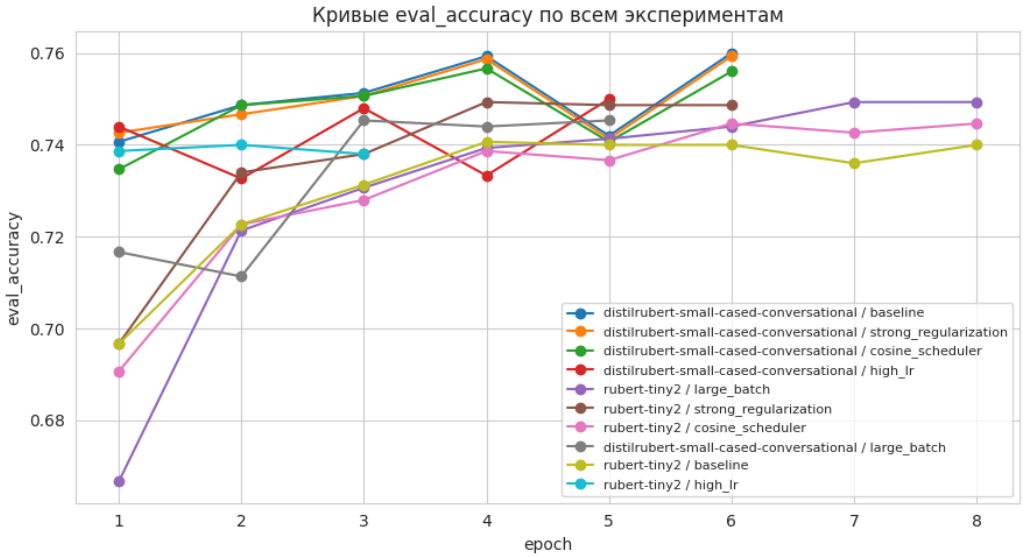


Рисунок 4 — Динамика eval_accuracy по эпохам для всех 10 экспериментов

Кривые eval_accuracy качественно повторяют картину eval_macro_f1 (что ожидаемо для сбалансированного датасета). Это подтверждает корректность выбора macro F1 в качестве главной метрики: на сбалансированных данных она согласована с accuracy, но более чувствительна к качеству по отдельным классам.

Механизм ранней остановки сработал по-разному для разных конфигураций. Распределение конфигураций по числу пройденных эпох представлено в Таблице 13.

Таблица 13 – Распределение конфигураций по числу пройденных эпох

Эпох пройдено	**Конфигурации**
3	rubert-tiny2 / high_lr
5	distilrubert / high_lr, distilrubert / large_batch
6	distilrubert / baseline, distilrubert / strong_reg, distilrubert / cosine, rubert-tiny2 / strong_reg
8 (max)	rubert-tiny2 / baseline, rubert-tiny2 / large_batch, rubert-tiny2 / cosine

Данные результаты остановок можно интерпретировать следующим образом:

- distilRuBERT-конфигурации в среднем сходятся быстрее (5–6 эпох) — крупная модель быстрее извлекает информацию из обучающей выборки;

- ruBERT-tiny2 в большинстве случаев дошла до максимального лимита 8 эпох — это означает, что компактная модель не успела достичь полного насыщения при текущем числе эпох, и потенциально могла бы показать более высокое качество при увеличении бюджета обучения;
- high_lr на обеих моделях приводит к самой ранней остановке (3 и 5 эпох соответственно) — модель быстро деградирует, и early stopping корректно прерывает обучение.

Хотя eval_loss в ряде конфигураций начинает расти после 3-й эпохи, macro F1 продолжает медленно улучшаться или остаётся стабильным. Это указывает на то, что модель учится корректно классифицировать примеры, но становится «менее уверенной» в своих предсказаниях. Для прикладной задачи, где важна именно классификация, а не калибровка вероятностей, это поведение приемлемо. Механизм load_best_model_at_end=True в сочетании с metric_for_best_model='eval_macro_f1' гарантирует, что для дальнейшего использования будет восстановлен чекпойнт с лучшим F1, а не с минимальным loss.

3.7 Сравнение результатов экспериментов и выбор лучшей модели

По итогам сравнения 10 дообученных моделей и 2 готовых pretrained-моделей построен общий рейтинг представленный на Рисунке 5.

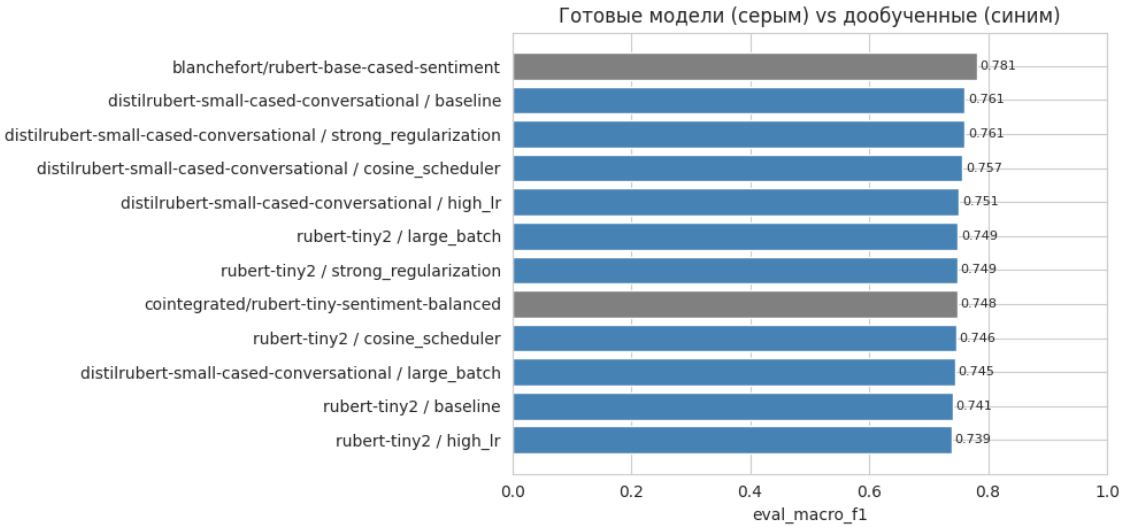


Рисунок 5 — Сравнение всех моделей

На графике приведены значения macro F1 для всех 12 моделей, отсортированных по убыванию. Видно, что:

- готовая модель blanchefort/rubert-base-cased-sentiment (0.781) занимает первое место, опережая все наши дообученные конфигурации;
- топ-4 дообученных моделей на базе DistilRuBERT (0.745–0.761) занимают позиции 2–5;
- готовая модель cointegrated/rubert-tiny-sentiment-balanced (0.748) располагается в середине списка дообученных моделей, на уровне лучших конфигураций RuBERT-tiny2.
- Дообученные конфигурации RuBERT-tiny2 (0.739–0.750) занимают нижнюю часть рейтинга.

Отставание наших дообученных моделей от готовой blanchefort/rubert-base-cased-sentiment обусловлено размером базовой модели и объемом обучающих данных. blanchefort основана на полноразмерной RuBERT-base (~178M параметров), тогда как наши модели существенно меньше (DistilRuBERT — 107 M, RuBERT-tiny2 — 29 M). Также автор blanchefort/rubert-base-cased-sentiment дообучал модель на больших корпусах русскоязычных отзывов из нескольких источников, тогда как мы использовали лишь небольшое количество примеров.

Вместе с тем, наши результаты демонстрируют ценные практические выводы: компактная модель DistilRuBERT, дообученная всего на 6 000 примерах, достигает 97 % качества полноразмерной готовой модели, обладая при этом существенно меньшим размером и более высокой скоростью инференса. Сравнение с cointegrated/rubert-tiny-sentiment-balanced показывает, что наши лучшие конфигурации DistilRuBERT уверенно превосходят готовую компактную модель (0.761 vs 0.748), что подтверждает целесообразность самостоятельного дообучения для специфичного домена.

В качестве лучшей модели выбрана конфигурация DistilRuBERT-small-cased-conversational / baseline. Выбор baseline обусловлен принципом бритвы

Оккама: при равном качестве предпочтение отдаётся более простой конфигурации со стандартными значениями гиперпараметров, которые лучше переносятся на новые задачи.

3.8 Анализ ошибок классификации и матрица ошибок

Для углублённого анализа поведения лучшей модели построена матрица ошибок и рассчитаны метрики по каждому классу.

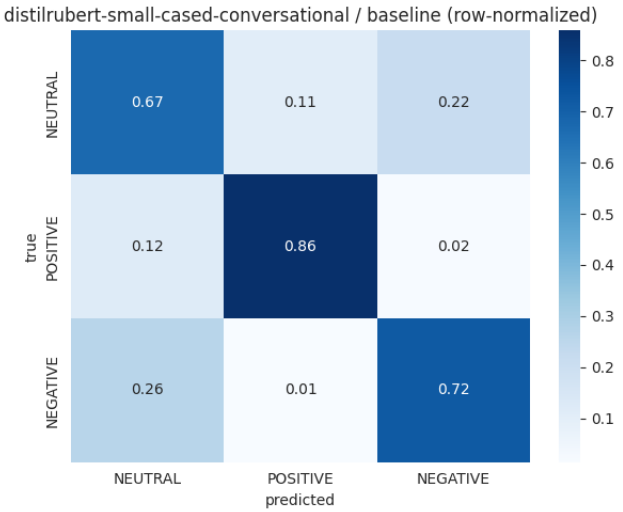


Рисунок 6 — Матрица ошибок лучшей модели

Из матрицы ошибок видны следующие закономерности:

- POSITIVE — самый «лёгкий» класс: 86 % примеров классифицированы корректно. Модель практически никогда не путает позитивные отзывы с негативными (всего 2 % ошибок). Это объясняется тем, что позитивная лексика в отзывах является наиболее однозначной: «отличный», «нравится», «рекомендую», «довольна»;
- NEGATIVE классифицируется уверенно (72 %), и почти никогда не путается с POSITIVE (1 %). Однако 26 % негативных отзывов модель относит к нейтральному классу — это основной источник ошибок для негативного класса;
- NEUTRAL — самый сложный класс (только 67 % корректных предсказаний). При этом 22 % нейтральных отзывов модель относит к NEGATIVE и 11 % — к POSITIVE;

- главные оси путаницы — между нейтральным и эмоционально окрашенными классами;
- разделение POSITIVE/NEGATIVE работает отлично: всего 1.7 % ошибок из всех примеров двух эмоционально окрашенных классов. Модель уверенно различает «положительные» и «отрицательные» отзывы — затруднения возникают лишь при определении границы между «оценочным» и «нейтральным» текстом.

Для лучшей модели был составлен столбчатый график, который показывает метрики модели по каждому классу. На Рисунке 7 представлены метрики по классам лучшей модели.

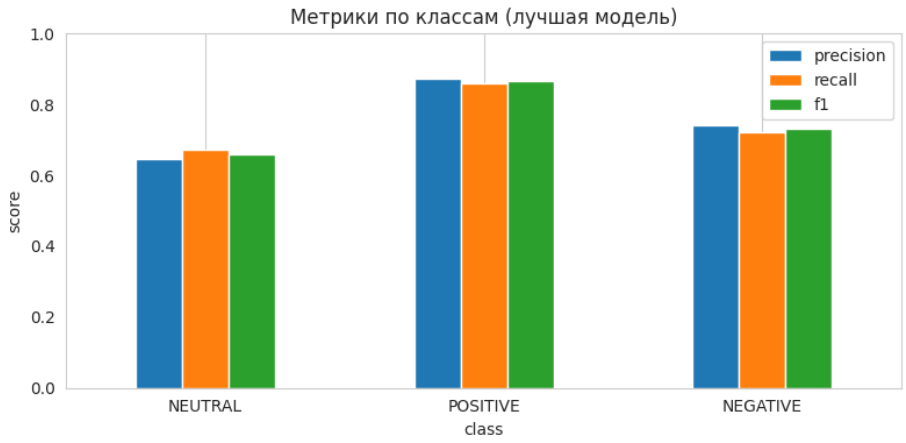


Рисунок 7 — Метрики по классам для лучшей модели

Из рисунка видна явная иерархия качества по классам:

- POSITIVE: F1 = 0.866 — наилучший показатель, сбалансированные precision и recall.
- NEGATIVE: F1 = 0.731 — приемлемое качество с лёгким перекосом в сторону precision (модель чуть «осторожнее» в назначении класса NEGATIVE).
- NEUTRAL: F1 = 0.659 — самое низкое качество. Низкая precision (0.645) указывает на то, что модель часто относит к нейтральному классу примеры из эмоционально окрашенных классов.

Также для качественного анализа предсказаний модель была протестирована на отдельных текстах. Результаты предсказаний представлены в Таблице 14.

Таблица 14 – Примеры предсказаний лучшей модели

№	Текст отзыва	Предсказание	Уверенность
1	«Платье отличное, ткань приятная, села по фигуре, очень довольна покупкой!»	POSITIVE	0.998
2	«Сумка пришла с оторванной ручкой, ужасное качество, деньги на ветер.»	NEGATIVE	0.972
3	«Заказ доставили вовремя, размер совпал с таблицей, в целом нормально.»	POSITIVE	0.940
4	«Туфли красивые, но натирают пятку после получаса носки.»	NEUTRAL	0.861
5	«Шарф мягкий и тёплый, цвет как на фото, рекомендую продавца.»	POSITIVE	0.997

Примеры 1, 2, 5 — однозначно эмоциональные отзывы — классифицированы с очень высокой уверенностью (>0.97). Это типичный случай корректной работы модели на «лёгких» примерах.

Пример 3 «нормально» — субъективно нейтральный отзыв («в целом нормально»), но модель уверенно отнесла к POSITIVE (0.94). Это пограничный случай, и предсказание спорное — слово «нормально» в русском языке часто несёт слабоположительную окраску, но эталонная разметка могла бы быть и NEUTRAL.

Пример 4 «но натирают пятку» — отзыв со смешанной тональностью (положительная оценка внешнего вида + отрицательная оценка комфорта). Модель выбрала NEUTRAL, что является разумным компромиссом для документ-уровневой классификации. Подобные смешанные случаи — типичный источник ошибок и теоретически требуют перехода на аспектный анализ тональности.

Анализ матрицы ошибок и примеров позволяет выделить главные источники ошибок лучшей модели:

- смешанные отзывы — содержат и положительные, и отрицательные элементы (как пример 4). Документ-уровневая модель вынуждена выбирать один класс из трёх, что приводит к ошибкам разметки;
- слабо-эмоциональные отзывы — содержат вежливые формулировки вроде «нормально», «неплохо», «терпимо», которые человек может разметить как нейтральные, а модель — как слабо-позитивные/негативные;
- отсутствие явных эмоциональных маркеров в нейтральных описательных отзывах — например, перечисление фактов о доставке без оценки.

В третьей главе представлены результаты практической части исследования. Реализован полный пайплайн обучения и оценки моделей классификации тональности на базе экосистемы transformers. Проведено сравнение двух готовых pretrained-моделей. Выполнены 10 экспериментов по дообучению. Все дообученные модели показали macro F1 в диапазоне 0.739–0.761, что свидетельствует об устойчивости задачи к выбору гиперпараметров в исследуемом диапазоне. Лучшая модель — DistilRuBERT-small-cased-conversational в конфигурации baseline с macro F1 = 0.7609 и accuracy = 0.760. Анализ ошибок показал, что главным источником ошибок является путаница между нейтральным классом и эмоционально окрашенными. При этом разделение POSITIVE/NEGATIVE работает практически безошибочно. Сравнение с готовой моделью показало, что дообучение компактных моделей на ограниченной выборке достигает 97 % качества полноразмерной готовой модели с учетом специфики данных.

ЗАКЛЮЧЕНИЕ

В рамках курсовой работы решена задача классификации тональности русскоязычных текстов отзывов с применением предобученных трансформерных моделей и методики дообучения. Все поставленные цели и задачи выполнены в полном объёме.

В первой главе рассмотрены задача анализа тональности, основные подходы к её решению, архитектура Transformer и модель BERT, а также её русскоязычные адаптации — RuBERT, DistilRuBERT и rubert-tiny2. Обоснован выбор трансформерных моделей как наиболее эффективного класса решений для задачи классификации тональности на современном этапе развития методов обработки естественного языка.

Во второй главе обоснован выбор открытого сбалансированного корпуса RuReviews, описана процедура минимальной предобработки данных. Описана архитектура классификационной модели на базе предобученного трансформера. Обоснован выбор macro F1 в качестве главной метрики оценки качества. Сформирован план исследования из 10 экспериментов, охватывающего 4 ключевые оси гиперпараметров.

В третьей главе на основе экосистемы библиотек Python реализован унифицированный пайплайн дообучения и оценки моделей. Унификация процедуры обеспечила корректность сравнения экспериментов: различия в финальных метриках обусловлены исключительно различиями в значениях исследуемых гиперпараметров.

Лучшие результаты показала модель DistilRuBERT-small-cased-conversational в базовой конфигурации. Установлено, что главным источником ошибок является путаница между нейтральным классом и эмоционально окрашенными классами.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Анализ тональности текста — Википедия [Электронный ресурс] — URL: https://ru.wikipedia.org/wiki/Анализ_тональности_текста (дата обращения: 25.04.2026).

2. Vaswani A., Shazeer N., Parmar N., Uszkoreit J., Jones L., Gomez A. N., Kaiser, Polosukhin I. Attention Is All You Need [Электронный ресурс] — URL: https://arxiv.org/abs/1706.03762 (дата обращения: 25.04.2026).

3. Devlin J., Chang M.-W., Lee K., Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding [Электронный ресурс] — URL: https://arxiv.org/abs/1810.04805 (дата обращения: 25.04.2026).

4. Sanh V., Debut L., Chaumond J., Wolf T. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter [Электронный ресурс] — URL: https://arxiv.org/abs/1910.01108 (дата обращения: 25.04.2026).

5. Liu Y., Ott M., Goyal N., Du J., Joshi M., Chen D., Levy O., Lewis M., Zettlemoyer L., Stoyanov V. RoBERTa: A Robustly Optimized BERT Pretraining Approach [Электронный ресурс] — URL: https://arxiv.org/abs/1907.11692 (дата обращения: 25.04.2026).

6. Kuratov Y., Arkhipov M. Adaptation of Deep Bidirectional Multilingual Transformers for Russian Language [Электронный ресурс] — URL: https://arxiv.org/abs/1905.07213 (дата обращения: 25.04.2026).

7. DeepPavlov: Open-source библиотека для разработки диалоговых систем [Электронный ресурс] — URL: https://deeppavlov.ai/ (дата обращения: 25.04.2026).

8. Smetanin S., Komarov M. Deep transfer learning baselines for sentiment analysis in Russian [Электронный ресурс] — URL: https://www.sciencedirect.com/science/article/abs/pii/S0306457321000236 (дата обращения: 25.04.2026).

9. RuReviews: датасет отзывов на русском языке для задачи классификации тональности [Электронный ресурс] — URL: https://github.com/sismetanin/rureviews (дата обращения: 25.04.2026).

10. Hugging Face Transformers: State-of-the-art Machine Learning for PyTorch, TensorFlow, and JAX [Электронный ресурс] — URL: https://huggingface.co/docs/transformers (дата обращения: 25.04.2026).

11. Модель blanchefort/rubert-base-cased-sentiment на Hugging Face Hub [Электронный ресурс] — URL: https://huggingface.co/blanchefort/rubert-base-cased-sentiment (дата обращения: 26.04.2026).

12. Модель cointegrated/rubert-tiny2 на Hugging Face Hub [Электронный ресурс] — URL: https://huggingface.co/cointegrated/rubert-tiny2 (дата обращения: 25.04.2026).

13. Модель cointegrated/rubert-tiny-sentiment-balanced на Hugging Face Hub [Электронный ресурс] — URL: https://huggingface.co/cointegrated/rubert-tiny-sentiment-balanced (дата обращения: 26.04.2026).

14. Модель DeepPavlov/distilrubert-small-cased-conversational на Hugging Face Hub [Электронный ресурс] — URL: https://huggingface.co/DeepPavlov/distilrubert-small-cased-conversational (дата обращения: 26.04.2026).

15. Loshchilov I., Hutter F. Decoupled Weight Decay Regularization (AdamW) [Электронный ресурс] — URL: https://arxiv.org/abs/1711.05101 (дата обращения: 27.04.2026).

16. Loshchilov I., Hutter F. SGDR: Stochastic Gradient Descent with Warm Restarts (Cosine Scheduler) [Электронный ресурс] — URL: https://arxiv.org/abs/1608.03983 (дата обращения: 27.04.2026).

17. Sun C., Qiu X., Xu Y., Huang X. How to Fine-Tune BERT for Text Classification? [Электронный ресурс] — URL: https://arxiv.org/abs/1905.05583 (дата обращения: 27.04.2026).

18. Mosolova A., Fomin V., Bondarenko I. Text Augmentation for Neural Networks [Электронный ресурс] — URL: https://arxiv.org/abs/1812.01401 (дата обращения: 27.04.2026).

19. Sennrich R., Haddow B., Birch A. Neural Machine Translation of Rare Words with Subword Units (BPE) [Электронный ресурс] — URL: https://arxiv.org/abs/1508.07909 (дата обращения: 28.04.2026).

20. Pang B., Lee L. Opinion Mining and Sentiment Analysis [Электронный ресурс] — URL: https://www.cs.cornell.edu/home/llee/omsa/omsa.pdf (дата обращения: 28.04.2026).

21. Liu B. Sentiment Analysis and Opinion Mining // Synthesis Lectures on Human Language Technologies — 2012 [Электронный ресурс] — URL: https://www.cs.uic.edu/~liub/FBS/SentimentAnalysis-and-OpinionMining.pdf (дата обращения: 28.04.2026).

22. Scikit-learn: Machine Learning in Python [Электронный ресурс] — URL: https://scikit-learn.org/stable/ (дата обращения: 28.04.2026).

23. PyTorch: An Open Source Machine Learning Framework [Электронный ресурс] — URL: https://pytorch.org/ (дата обращения: 28.04.2026).

24. Hugging Face Datasets: A community library for natural language processing [Электронный ресурс] — URL: https://huggingface.co/docs/datasets (дата обращения: 29.04.2026).

25. Google Colaboratory — облачная среда исполнения Python-кода [Электронный ресурс] — URL: https://colab.research.google.com/ (дата обращения: 30.04.2026).

Приложение

Листинг 1 – Импорты и константы

```
import os
import gc
import json
import shutil

import numpy as np
import pandas as pd
import torch

from datasets import Dataset, DatasetDict
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    DataCollatorWithPadding,
    TrainingArguments,
    Trainer,
    EarlyStoppingCallback,
    pipeline,
)
from sklearn.metrics import accuracy_score, precision_recall_fscore_support

SEED = 42
DEVICE = 0 if torch.cuda.is_available() else -1

np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)

print('DEVICE:', DEVICE, '| CUDA:', torch.cuda.is_available())
```

Листинг 2 – Загрузка датасета

```
DATASET_URL = 'https://raw.githubusercontent.com/sismetanin/rureviews/master/women-clothing-accessories.3-class.balanced.csv'
df = pd.read_csv(DATASET_URL, sep='\t')
print('Размер:', df.shape)
df.head()
```

Листинг 3 – Приведение к единому виду

```
df = df.rename(columns={'review': 'text', 'sentiment': 'label'})
label_map_if_str = {
    'neutral': 0, 'нейтрально': 0, 'neautral': 0,
    'positive': 1, 'позитив': 1, 'положительно': 1,
    'negative': 2, 'негатив': 2, 'отрицательно': 2,
}
def cast_label(y):
    if isinstance(y, str):
        return label_map_if_str[y.strip().lower()]
    return int(y)
df['text'] = df['text'].astype(str)
df['label'] = df['label'].apply(cast_label)
print(df['label'].value_counts().sort_index())
df.head()
```

Листинг 4 – Стратифицированное деление на train / validation / test

```
df = df.sample(frac=1, random_state=SEED).reset_index(drop=True)

val_parts, test_parts, train_parts = [], [], []
for cls, grp in df.groupby('label'):
    n_val  = int(len(grp) * 0.15)
```

```
    n_test = int(len(grp) * 0.15)
    val_parts.append(grp.iloc[:n_val])
    test_parts.append(grp.iloc[n_val:n_val + n_test])
    train_parts.append(grp.iloc[n_val + n_test:])

train_df = pd.concat(train_parts).sample(frac=1, random_state=SEED).reset_inde
x(drop=True)
valid_df = pd.concat(val_parts).sample(frac=1, random_state=SEED).reset_index(d
rop=True)
test_df  = pd.concat(test_parts).sample(frac=1, random_state=SEED).reset_index
(drop=True)

raw = DatasetDict({
    'train': Dataset.from_pandas(train_df[['text', 'label']], preserve_index=F
alse),
    'validation': Dataset.from_pandas(valid_df[['text', 'label']], preserve_in
dex=False),
    'test': Dataset.from_pandas(test_df[['text', 'label']], preserve_index=Fal
se),
})
raw
```

Листинг 5 – Ограничение размера выборки

```
MAX_TRAIN = 60000
MAX_VALID = 15000
MAX_TEST  = 15000

train_ds = raw['train'].shuffle(seed=SEED)
valid_ds = raw['validation'].shuffle(seed=SEED)
test_ds  = raw['test'].shuffle(seed=SEED)

if MAX_TRAIN is not None:
    train_ds = train_ds.select(range(min(MAX_TRAIN, len(train_ds))))
if MAX_VALID is not None:
    valid_ds = valid_ds.select(range(min(MAX_VALID, len(valid_ds))))
if MAX_TEST is not None:
    test_ds  = test_ds.select(range(min(MAX_TEST, len(test_ds))))

dataset = DatasetDict({'train': train_ds, 'validation': valid_ds, 'test': test
_ds})
print(dataset)
print(pd.Series(dataset['train']['label']).value_counts().sort_index())
print(pd.Series(dataset['validation']['label']).value_counts().sort_index())
print(pd.Series(dataset['test']['label']).value_counts().sort_index())
```

Листинг 6 – Маппинги меток и примеры для проверки моделей

```
ID2LABEL = {0: 'NEUTRAL', 1: 'POSITIVE', 2: 'NEGATIVE'}
LABEL2ID = {v: k for k, v in ID2LABEL.items()}
examples = [
    'Платье отличное, ткань приятная, села по фигуре, очень довольна покупкой!
',
    'Сумка пришла с оторванной ручкой, ужасное качество, деньги на ветер.',
    'Заказ доставили вовремя, размер совпал с таблицей, в целом нормально.',
    'Туфли красивые, но натирают пятку после получаса носки.',
    'Шарф мягкий и тёплый, цвет как на фото, рекомендую продавца.',
]
```

Листинг 7 – Готовые модели через pipeline

```
PIPELINE_MODELS = [
    'blanchefort/rubert-base-cased-sentiment',
    'cointegrated/rubert-tiny-sentiment-balanced',
]

for model_name in PIPELINE_MODELS:
    print('MODEL:', model_name)
    clf = pipeline(
```

```
        'sentiment-analysis',
        model=model_name,
        tokenizer=model_name,
        device=DEVICE,
        truncation=True,
    )
    for text in examples:
        print(text, '->', clf(text))
```

Листинг 8 – Поэтапное предсказание без pipeline

```
def predict_step_by_step(model_name, texts):
    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = AutoModelForSequenceClassification.from_pretrained(model_name)
    model.eval()
    if torch.cuda.is_available():
        model.cuda()

    batch = tokenizer(
        texts, padding=True, truncation=True, max_length=256, return_tensors='
pt'
    )
    if torch.cuda.is_available():
        batch = {k: v.cuda() for k, v in batch.items()}

    with torch.no_grad():
        logits = model(**batch).logits
        probs = torch.softmax(logits, dim=-1).detach().cpu().numpy()

    id2label = model.config.id2label
    result = []
    for text, p in zip(texts, probs):
        pred_id = int(np.argmax(p))
        result.append({
            'text': text,
            'label': id2label.get(pred_id, str(pred_id)),
            'score': float(p[pred_id]),
            'all_probabilities': p.tolist(),
        })

    del model
    if torch.cuda.is_available():
        torch.cuda.empty_cache()
    return result

for model_name in PIPELINE_MODELS:
    print('MODEL:', model_name)
    preds = predict_step_by_step(model_name, examples)
    for row in preds:
        print(row)
```

Листинг 9 – Оценка готовых моделей на валидации

```
def normalize_pred_label(label):
    label = str(label).upper().strip()
    label_map = {
        'NEUTRAL': 0, 'LABEL_0': 0, '0': 0,
        'POSITIVE': 1, 'LABEL_1': 1, '1': 1,
        'NEGATIVE': 2, 'LABEL_2': 2, '2': 2,
    }
    if label not in label_map:
        raise ValueError(f'Неизвестная метка модели: {label}')
    return label_map[label]
```

```
PIPELINE_EVAL_SIZE = min(500, len(dataset['validation']))
eval_part = dataset['validation'].select(range(PIPELINE_EVAL_SIZE))

eval_texts = ['' if x is None else str(x) for x in eval_part['text']]
y_true = np.array([int(y) for y in eval_part['label']])

pipeline_results = []
for model_name in PIPELINE_MODELS:
    print(f'\nМодель: {model_name}')
    clf = pipeline(
        'sentiment-analysis',
        model=model_name,
        tokenizer=model_name,
        device=DEVICE,
        truncation=True,
    )
    preds = clf(eval_texts, batch_size=32, max_length=256, truncation=True)
    y_pred = np.array([normalize_pred_label(p['label']) for p in preds])

    acc = accuracy_score(y_true, y_pred)
    precision, recall, f1, _ = precision_recall_fscore_support(
        y_true, y_pred, average='macro', zero_division=0
    )
    pipeline_results.append({
        'model': model_name,
        'accuracy': acc,
        'macro_precision': precision,
        'macro_recall': recall,
        'macro_f1': f1,
    })

pipeline_results_df = pd.DataFrame(pipeline_results)
pipeline_results_df
```

Листинг 10 – Конфиги экспериментов для дообучения

```
FINETUNE_MODELS = [
    'cointegrated/rubert-tiny2',
    'DeepPavlov/distilrubert-small-cased-conversational',
]

BASE_CFG = {
    'num_train_epochs': 20,
    'learning_rate': 2e-5,
    'lr_scheduler_type': 'linear',
    'optim': 'adamw_torch',
    'per_device_train_batch_size': 16,
```

```
    'per_device_eval_batch_size': 16,
    'gradient_accumulation_steps': 1,
    'weight_decay': 0.01,
    'warmup_ratio': 0.1,
    'metric_for_best_model': 'macro_f1',
    'fp16': True,
    'early_stopping': True,
    'early_stopping_patience': 2,
    'greater_is_better': True,
}

EXPERIMENT_CONFIGS = [
    # 1) Baseline — точка отсчёта
    {**BASE_CFG, 'name': 'baseline'},

    # 2) Влияние learning rate: 2e-5 -> 5e-5
    {**BASE_CFG, 'name': 'high_lr',
     'learning_rate': 5e-5},

    # 3) Влияние lr scheduler: linear -> cosine
    {**BASE_CFG, 'name': 'cosine_scheduler',
     'lr_scheduler_type': 'cosine'},

    # 4) Влияние регуляризации: wd 0.01 -> 0.1
    {**BASE_CFG, 'name': 'strong_regularization',
     'weight_decay': 0.1},

    # 5) Влияние размера батча: 16 -> 32
    {**BASE_CFG, 'name': 'large_batch',
     'per_device_train_batch_size': 32,
     'per_device_eval_batch_size': 32},
]

RUN_ALL_CONFIGS = True
configs_to_run = EXPERIMENT_CONFIGS if RUN_ALL_CONFIGS else EXPERIMENT_CONFIGS
[:1]
configs_to_run
```

Листинг 11 – Утилиты: метрики, токенизация, TrainingArguments

```
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    if isinstance(logits, tuple):
        logits = logits[0]
    preds = np.argmax(logits, axis=-1)
    acc = accuracy_score(labels, preds)
    pr, rc, f1, _ = precision_recall_fscore_support(
        labels, preds, average='macro', zero_division=0
    )
    return {
        'accuracy': acc,
        'macro_precision': pr,
```

```
        'macro_recall': rc,
        'macro_f1': f1,
    }


def tokenize_dataset(model_name):
    tokenizer = AutoTokenizer.from_pretrained(model_name)

    def tok(batch):
        return tokenizer(batch['text'], truncation=True, max_length=256)

    keep = {'label'}
    remove_cols = [c for c in dataset['train'].column_names if c not in keep]
    tokenized = dataset.map(tok, batched=True, remove_columns=remove_cols)

    data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
    return tokenizer, tokenized, data_collator


def make_training_args(output_dir, cfg):
    common = dict(
        output_dir=output_dir,
        num_train_epochs=cfg['num_train_epochs'],
        learning_rate=cfg['learning_rate'],
        lr_scheduler_type=cfg['lr_scheduler_type'],
        optim=cfg['optim'],
        per_device_train_batch_size=cfg['per_device_train_batch_size'],
        per_device_eval_batch_size=cfg['per_device_eval_batch_size'],
        gradient_accumulation_steps=cfg['gradient_accumulation_steps'],
        fp16=bool(cfg['fp16'] and torch.cuda.is_available()),
        logging_strategy='steps',
        logging_steps=25,
        save_strategy='epoch',
        load_best_model_at_end=True,
        metric_for_best_model=cfg['metric_for_best_model'],
        greater_is_better=cfg['greater_is_better'],
        save_total_limit=2,
        report_to='none',
        seed=SEED,
        warmup_ratio=cfg.get('warmup_ratio', 0.1),
        weight_decay=cfg.get('weight_decay', 0.01),
        max_grad_norm=1.0,
        dataloader_num_workers=2,
        group_by_length=True,
    )

    try:
        return TrainingArguments(eval_strategy='epoch', **common)
    except TypeError:
        return TrainingArguments(evaluation_strategy='epoch', **common)
```

Листинг 12 – Запуск одного эксперимента

```
def train_one_experiment(model_name, cfg):
    safe_model_name = model_name.replace('/', '__')
    output_dir = f'./results/{safe_model_name}/{cfg["name"]}'
    shutil.rmtree(output_dir, ignore_errors=True)
    os.makedirs(output_dir, exist_ok=True)

    tokenizer, tokenized, data_collator = tokenize_dataset(model_name)
    model = AutoModelForSequenceClassification.from_pretrained(
        model_name,
        num_labels=3,
        id2label=ID2LABEL,
        label2id=LABEL2ID,
```

```
        ignore_mismatched_sizes=True,
    )

    args = make_training_args(output_dir, cfg)

    callbacks = []
    if cfg.get('early_stopping'):
        patience = cfg.get('early_stopping_patience', 2)
        callbacks.append(EarlyStoppingCallback(early_stopping_patience=patienc
e))

    trainer_kwargs = dict(
        model=model,
        args=args,
        train_dataset=tokenized['train'],
        eval_dataset=tokenized['validation'],
        data_collator=data_collator,
        compute_metrics=compute_metrics,
        callbacks=callbacks,
    )

    try:
        trainer = Trainer(processing_class=tokenizer, **trainer_kwargs)
    except TypeError:
        trainer = Trainer(tokenizer=tokenizer, **trainer_kwargs)

    trainer.train()
    metrics = trainer.evaluate()

    history_path = os.path.join(output_dir, 'trainer_state_log_history.json')
    with open(history_path, 'w', encoding='utf-8') as f:
        json.dump(trainer.state.log_history, f, ensure_ascii=False, indent=2)

    result = {
        'base_model': model_name,
        'config_name': cfg['name'],
        **{k: v for k, v in cfg.items() if k != 'name'},
        **metrics,
        'output_dir': output_dir,
    }

    del trainer, model
    gc.collect()
    if torch.cuda.is_available():
        torch.cuda.empty_cache()

    return result
```

Листинг 13 – Перебор моделей и конфигов

```
all_results = []

for model_name in FINETUNE_MODELS:
    for cfg in configs_to_run:
        print('=' * 100)
        print('MODEL:', model_name)
        print('CONFIG:', cfg['name'])
        try:
            res = train_one_experiment(model_name, cfg)
            all_results.append(res)
            print(res)
```

```
        except RuntimeError as e:
            print('RuntimeError:', e)
            print('Если не хватает памяти — уменьшите MAX_TRAIN, MAX_VALID или
batch size.')
            gc.collect()
            if torch.cuda.is_available():
                torch.cuda.empty_cache()

results_df = pd.DataFrame(all_results)
results_df = results_df.sort_values(['eval_macro_f1', 'eval_accuracy'], ascend
ing=False)
results_df

os.makedirs('/content/sentiment_results', exist_ok=True)
results_df.to_csv('/content/sentiment_results/finetune_results.csv',
index=False)
results_df
```

Листинг 14 – Загрузка истории + аккуратная фильтрация

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report

sns.set_style('whitegrid')

def load_history(output_dir):
    path = os.path.join(output_dir, 'trainer_state_log_history.json')
    with open(path, 'r', encoding='utf-8') as f:
        return pd.DataFrame(json.load(f))


def split_train_eval(hist):
    """
    Делит лог на train-шаги и eval-шаги.
    Убирает финальные сводные записи (train_runtime, train_loss без step)
    и дедуплицирует по epoch — оставляем последнюю запись на эпоху.
    """
    # Train-логи: есть 'loss', но это НЕ финальная сводка (у неё есть train_ru
ntime)
    if 'loss' in hist.columns:
        train_hist = hist[hist['loss'].notna()].copy()
        if 'train_runtime' in train_hist.columns:
            train_hist = train_hist[train_hist['train_runtime'].isna()]
    else:
        train_hist = pd.DataFrame()
```

Продолжение Листинга 14

```
    # Eval-логи
    if 'eval_loss' in hist.columns:
        eval_hist = hist[hist['eval_loss'].notna()].copy()
        if 'epoch' in eval_hist.columns:
            eval_hist = (eval_hist
                         .sort_index()
                         .drop_duplicates(subset='epoch', keep='last')
                         .sort_values('epoch')
                         .reset_index(drop=True))
    else:
        eval_hist = pd.DataFrame()

    if not train_hist.empty and 'step' in train_hist.columns:
```

```
        train_hist = train_hist.sort_values('step').reset_index(drop=True)

    return train_hist, eval_hist
```

Листинг 15 – Графики истории одного эксперимента

```
def plot_history(output_dir, title, x_axis='epoch'):
    """
    x_axis: 'epoch' (по умолчанию, целые отметки на eval) или 'step'.
    Train-кривая будет плотной (по logging_steps), eval-кривая — по одной точк
е на эпоху.
    """
    hist = load_history(output_dir)
    train_hist, eval_hist = split_train_eval(hist)

    x_col = x_axis if x_axis in ('epoch', 'step') else 'epoch'

    # 1. Loss: train vs eval
    plt.figure(figsize=(8, 4))

    # в ячейке 16, в блоке "1. Loss"
    if not train_hist.empty and 'loss' in train_hist.columns:
        plt.plot(train_hist[x_col], train_hist['loss'],
                 linestyle='-', linewidth=1, alpha=0.3, label='train loss (raw)
')
        # сглаженная версия
        smooth = train_hist['loss'].rolling(window=5, min_periods=1).mean()
        plt.plot(train_hist[x_col], smooth,
                 linestyle='-', linewidth=2, label='train loss (smoothed)')
    if not eval_hist.empty and 'eval_loss' in eval_hist.columns and x_col in e
val_hist.columns:
        plt.plot(eval_hist[x_col], eval_hist['eval_loss'],
                  marker='o', linewidth=2, label='eval loss')
    plt.title(f'{title}\nLoss')
    plt.xlabel(x_col); plt.ylabel('loss'); plt.legend(); plt.grid(True)
    plt.show()

    # 2. Eval-метрики
    metric_cols = [c for c in ['eval_accuracy', 'eval_macro_f1',
                               'eval_macro_precision', 'eval_macro_recall']
                   if c in eval_hist.columns]
    if metric_cols:
        plt.ylim(0.5, 1.0)
        plt.figure(figsize=(8, 4))
        for m in metric_cols:
            plt.plot(eval_hist[x_col], eval_hist[m], marker='o', label=m)
```

Продолжение Листинга 15

```
        plt.title(f'{title}\nEval metrics')
        plt.xlabel(x_col); plt.ylabel('score'); plt.legend(); plt.grid(True)
        plt.show()

    # 3. Learning rate
    if 'learning_rate' in train_hist.columns and train_hist['learning_rate'].n
otna().any():
        plt.figure(figsize=(8, 3))
        plt.plot(train_hist[x_col],  train_hist['learning_rate'],  linewidth=1.
5)
        # warmup_ratio=0.1 -> вертикальная линия на 10% от max epoch
        if x_col == 'epoch':
            max_epoch = train_hist['epoch'].max()
            plt.axvline(max_epoch * 0.1, color='red', linestyle='--',
                        alpha=0.5, label='end of warmup')
```

```
            plt.legend()
        plt.title(f'{title}\nLearning rate')
        plt.xlabel(x_col); plt.ylabel('lr'); plt.grid(True)
        plt.show()

    # 4. Grad norm
    if 'grad_norm' in train_hist.columns and train_hist['grad_norm'].notna().a
ny():
        plt.figure(figsize=(8, 3))
        plt.plot(train_hist[x_col], train_hist['grad_norm'], linewidth=1)
        plt.title(f'{title}\nGradient norm')
        plt.xlabel(x_col); plt.ylabel('grad norm'); plt.grid(True)
        plt.show()
```

Листинг 16 – Сравнение всех экспериментов

```
def comparison_table(results_df):
    cols = ['base_model', 'config_name',
            'eval_accuracy', 'eval_macro_f1',
            'eval_macro_precision', 'eval_macro_recall',
            'eval_loss', 'eval_runtime']
    cols = [c for c in cols if c in results_df.columns]
    out = results_df[cols].copy().sort_values(
        ['eval_macro_f1', 'eval_accuracy'], ascending=False
    ).reset_index(drop=True)
    return out

comp_df = comparison_table(results_df)
comp_df
def comparison_with_delta(results_df, baseline_name='baseline'):
    cols = ['base_model', 'config_name',
            'eval_accuracy', 'eval_macro_f1', 'eval_loss']
    cols = [c for c in cols if c in results_df.columns]
    out = results_df[cols].copy()

    # Дельта к baseline в рамках одной модели
    deltas = []
    for base, sub in out.groupby('base_model'):
        base_row = sub[sub['config_name'] == baseline_name]
        if base_row.empty:
            sub['delta_f1'] = np.nan
            sub['delta_acc'] = np.nan
```

Продолжение Листинга 16

```
        else:
            f1_base = base_row['eval_macro_f1'].iloc[0]
            acc_base = base_row['eval_accuracy'].iloc[0]
            sub = sub.copy()
            sub['delta_f1'] = sub['eval_macro_f1'] - f1_base
            sub['delta_acc'] = sub['eval_accuracy'] - acc_base
        deltas.append(sub)

    return pd.concat(deltas).sort_values(
        ['base_model', 'eval_macro_f1'], ascending=[True, False]
    ).reset_index(drop=True)

comparison_with_delta(results_df)
```

Листинг 17 – Кривые обучения для всех экспериментов на одном поле

```
def plot_all_eval_curves(results_df, metric='eval_macro_f1'):
    plt.figure(figsize=(9, 5))
    for _, r in results_df.iterrows():
        try:
            hist = load_history(r['output_dir'])
            _, eval_hist = split_train_eval(hist)
            if metric in eval_hist.columns and eval_hist[metric].notna().any():
                label = f"{r['base_model'].split('/')[-1]} / {r['config_name']}
"
                plt.plot(eval_hist['epoch'], eval_hist[metric],
                         marker='o', label=label)
        except FileNotFoundError:
            continue
    plt.title(f'Кривые {metric} по всем экспериментам')
    plt.xlabel('epoch'); plt.ylabel(metric)
    plt.legend(fontsize=8, loc='best'); plt.grid(True); plt.tight_layout(); pl
t.show()


plot_all_eval_curves(results_df, 'eval_macro_f1')
plot_all_eval_curves(results_df, 'eval_loss')
plot_all_eval_curves(results_df, 'eval_accuracy')
```

Листинг 18 –Поиск лучшей модели

```
def resolve_best_checkpoint(output_dir):
    """Берём последний checkpoint-* или сам output_dir, если чекпойнтов нет."""
"
    if not os.path.isdir(output_dir):
        return output_dir
    checkpoints = [os.path.join(output_dir, d) for d in os.listdir(output_dir)
                   if d.startswith('checkpoint-')]
    if checkpoints:
        return sorted(checkpoints, key=lambda p: int(p.split('-')[-1]))[-1]
    return output_dir

best_row = results_df.sort_values(
    ['eval_macro_f1', 'eval_accuracy'], ascending=False
).iloc[0]
best_dir = best_row['output_dir']
best_model_path = resolve_best_checkpoint(best_dir)
```

Продолжение Листинга 18

```
print('Лучшая модель:', best_row['base_model'])
print('Конфигурация:', best_row['config_name'])
print('macro_f1 =', round(best_row['eval_macro_f1'], 4),
      '| accuracy =', round(best_row['eval_accuracy'], 4))
print('Каталог:', best_dir)
print('Путь к чекпойнту:', best_model_path)

plot_history(best_dir, f"{best_row['base_model']} / {best_row['config_name']}")
```

Листинг 19 – Матрица ошибок

```
def plot_confusion(y_true, y_pred, label_names, title='Confusion matrix'):
    cm = confusion_matrix(y_true, y_pred)
    cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True)

    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=label_names, yticklabels=label_names, ax=axes[0])
    axes[0].set_title(f'{title} (counts)')
```

```
    axes[0].set_xlabel('predicted'); axes[0].set_ylabel('true')

    sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='Blues',
                xticklabels=label_names, yticklabels=label_names, ax=axes[1])
    axes[1].set_title(f'{title} (row-normalized)')
    axes[1].set_xlabel('predicted'); axes[1].set_ylabel('true')

    plt.tight_layout(); plt.show()


plot_confusion(val_true, val_pred, label_names,
               title=f"{best_row['base_model'].split('/')[-1]} / {best_row['co
nfig_name']}")
```